

PART II — BUILDING THE INGESTION PIPELINE

Chapter 10

Putting Ingestion Together: ingest.py

"A pipeline is only as reliable as its most silently broken step."

Chapter 10 — Putting Ingestion Together: ingest.py

The previous three chapters built the individual organs of the ingestion pipeline: chunks emerged from Chapter 7, embedding vectors from Chapter 8, and the vector store from Chapter 9. This chapter assembles them into a single beating heart — **ingest.py** — the script you will run once when you first populate your knowledge base, and again every time a new document arrives. By the time you finish this chapter, you will be able to explain every line of that script, diagnose the most common failure modes, and extend it to new file types without breaking anything that already works.

There is a gap between "knowing the components" and "knowing how to wire them together reliably." That gap is where most real-world bugs live. A retriever that returns garbage is almost never a retriever bug — it is an ingestion bug that happened quietly three days earlier. Files double-ingested, chunks too short to carry meaning, a parent root that silently re-discovers every file the child roots already found — these are the failure modes this chapter is designed to help you avoid.

We will walk through each function in **ingest.py** in the order it executes: file discovery, loader routing, splitting, embedding, persisting, and the guards that make the whole thing safe to re-run without producing duplicates. Each section pairs the implementation with the reasoning — because the code is not interesting until you understand why each line is written the way it is.

10.1 File Discovery Across Multiple Roots

The first function in `ingest.py`, `discover_files()`, answers one question: which files should be ingested? The answer is never trivially "all files in a directory," because a real knowledge base spans multiple directories, file types, and subdirectory depths. What you need is a systematic walk — one that visits every candidate location, filters by supported extension, and returns a flat, deduplicated list of absolute paths.

Definition — SEARCH_ROOTS

A Python list of `pathlib.Path` objects that defines the set of directory trees the ingestion script will walk. Each root is traversed recursively using `Path.rglob("*")`. The list is ordered: files discovered through earlier roots appear earlier in the final list, which preserves a predictable ingestion sequence.

In the project, `SEARCH_ROOTS` is configured as seven distinct paths — one for each file type (pdfs, textfiles, csv, word, html, json) plus the `../data` root itself. This last entry is the catch-all for any file placed directly in the data directory without a type-specific subdirectory. The order matters: type-specific roots appear before the catch-all, so their files are discovered first and their paths land in the deduplication dictionary before the catch-all adds a second reference to the same physical file.

```
SEARCH_ROOTS = [
    Path("../data/pdfs"),
    Path("../data/textfiles"),
    Path("../data/csv"),
    Path("../data/word"),
    Path("../data/html"),
    Path("../data/json"),
    Path("../data"),          # catch-all — must come LAST
]

def discover_files() -> list[str]:
    file_paths = []
    for root in SEARCH_ROOTS:
        if root.exists():
            for p in root.rglob("*"):
                if p.is_file() and p.suffix.lower() in SUPPORTED_EXT:
                    file_paths.append(str(p))
    return list(dict.fromkeys(file_paths))  # deduplicate, preserve order
```

The deduplication mechanism is `dict.fromkeys()`. Because Python dictionaries maintain insertion order since 3.7, this one-liner both removes duplicate path strings and preserves the order in which they were first encountered. The first path string for any given file wins; all later duplicates of that same string are silently dropped. The result is a clean, ordered list of paths ready to be handed to the loader.

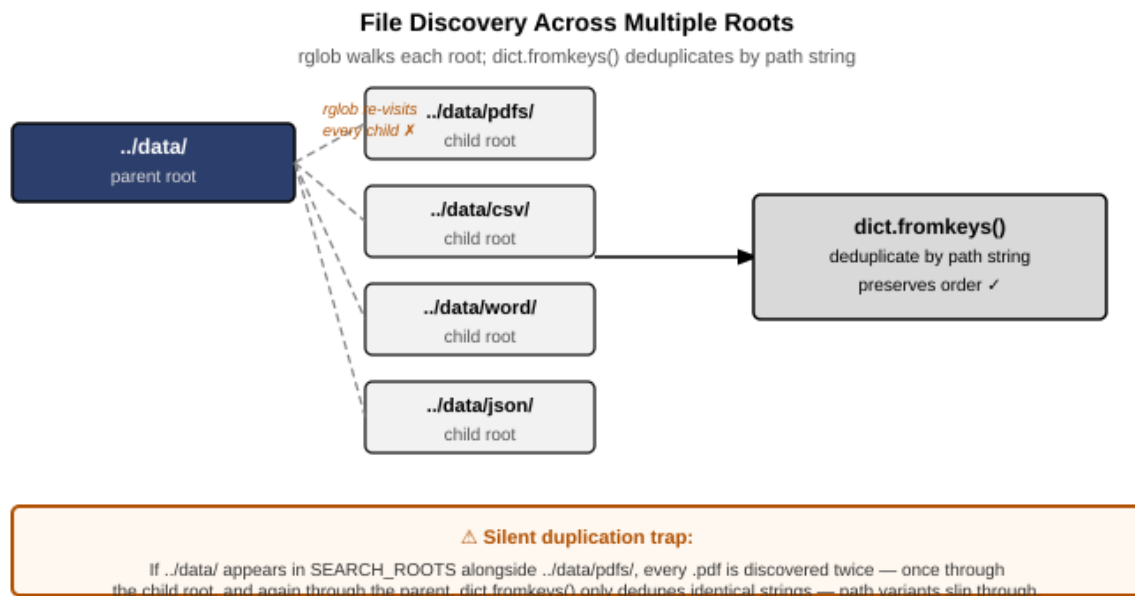


Figure 10.1 — File discovery across multiple roots. The catch-all `../data/` root at the end re-visits files already found by the child roots; `dict.fromkeys()` deduplicates by path string, but only if the path strings are identical.

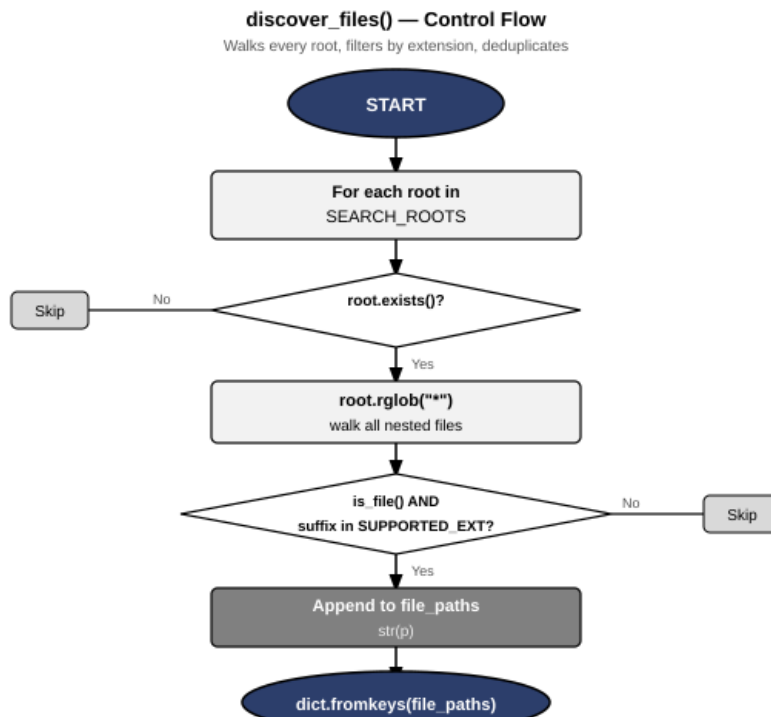


Figure 10.2 — `discover_files()` control flow. Each root is checked for existence before walking; unsupported extensions are skipped at the file level; `dict.fromkeys()` deduplicates the final list.

10.2 Avoiding Double-Ingestion

Section 10.1 described how `dict.fromkeys()` deduplicates by path string. That deduplication works correctly only if the same physical file produces the same string in every root it is discovered through. In practice, this condition is frequently violated — and when it is violated, you get silent duplication in your vector store.

Common pitfall — Parent root swallowing child roots

Consider this `SEARCH_ROOTS` list: `[Path("../data/pdfs"), Path("../data")]`. The file `"../data/pdfs/report.pdf"` is discovered first by the `pdfs` root, producing the string `"../data/pdfs/report.pdf"`. When the catch-all `../data` root runs, it walks recursively and also finds `"../data/pdfs/report.pdf"` — but on Windows, the path separator is a backslash. The two strings are not equal. `dict.fromkeys()` sees them as different keys, both survive, and the file is ingested twice, producing duplicate chunks in ChromaDB.

The fix is twofold: always place specific roots before the catch-all (so the first-seen string is the canonical one), and resolve paths to absolute form before comparing. Using `Path(fp).resolve()` before appending to `file_paths` collapses all path variants — relative vs absolute, forward- vs backslash, symlink vs real — into one canonical string.

A subtler variant of the same bug occurs when `../data` is listed *before* the type-specific subdirectories. In that case, the catch-all root discovers every file first, stamping the catch-all path string into the dictionary. The subsequent child-root walks find the same files but produce different strings (because the walk starts from a different base), which then fail the deduplication check. Every file ends up ingested twice. The correct ordering is always specific-before-general.

A more robust alternative — used in the companion code from Chapter 5 — is to deduplicate by OS inode number rather than by path string. The inode is the unique integer the operating system assigns to each physical file on disk. No matter how many names or paths point to the same file, its inode number is constant. The inode-based approach handles symlinks, relative-vs-absolute path variants, and case-insensitive filesystems all at once:

```
def discover_files_robust() -> list[str]:
    seen_inodes: set[int] = set()
    file_paths = []
    for root in SEARCH_ROOTS:
        if not root.exists():
            continue
        for p in root.rglob("*"):
            if not p.is_file():
                continue
            if p.suffix.lower() not in SUPPORTED_EXT:
                continue
            inode = p.stat().st_ino
            if inode in seen_inodes:
                continue # same physical file - skip
            seen_inodes.add(inode)
            file_paths.append(str(p))
    return file_paths
```

Analogy — The address book, not the street

Deduplicating by path string is like checking whether two people share the same mailing address. It works fine when addresses are written consistently. But if one person writes "123 Main St" and another writes "123 Main Street", the check fails even though both envelopes go to the same physical house. Deduplicating by inode is like checking whether two addresses share the same GPS coordinate. No matter how the address is written, the physical location is identical — and you catch the duplicate.

10.3 Routing Each File to the Right Loader

Once `discover_files()` has returned a flat list of paths, the next function — `load_documents()` — must decide which loader to use for each file. The decision is purely mechanical: it is a lookup from file extension to loading strategy. Three tracks exist in the current implementation.

Extension group	Loader	Output unit	Notes
.pdf, .txt, .doc, .docx, .html, .htm	UnstructuredLoader	1 Document per section/page	Batch-loads all paths in one call for efficiency
.csv, .xls, .xlsx	<code>_convert_tabular_to_json()</code> → JSONLoader	1 Document per row	Tabular files are first converted to a JSON records file, then loaded
.json	JSONLoader (<code>jq_schema="[]"</code>)	1 Document per array element	JSON arrays of record objects; schema configurable

The critical design choice is that `UnstructuredLoader` receives all unstructured paths in a single batch call, rather than one file at a time. Loading files one by one is dramatically slower because each call re-initialises the loader's internal state and re-imports its detection libraries. The batch call amortises that cost across the entire file set.

Tabular files require special handling because embedding a raw CSV row like `"patient_id,ethnicity,age,ASD_diagnosis"` produces a vector that represents the column-header syntax rather than the semantic content of the record. Converting to a JSON object first — `{"patient_id": 1, "ethnicity": "Caucasian", "age": 12, "ASD_diagnosis": true}` — preserves the field names alongside their values, giving the embedding model the context it needs to produce a meaningful vector.

Definition — Document object

The shared output type of every loader in `LangChain`. It carries exactly two fields: `page_content` (a string — the text that will be split and embedded) and `metadata` (a dictionary — the provenance labels that travel with each chunk through the pipeline). Every loader, regardless of format, produces `List[Document]`. This uniformity is what makes the rest of the pipeline format-agnostic.

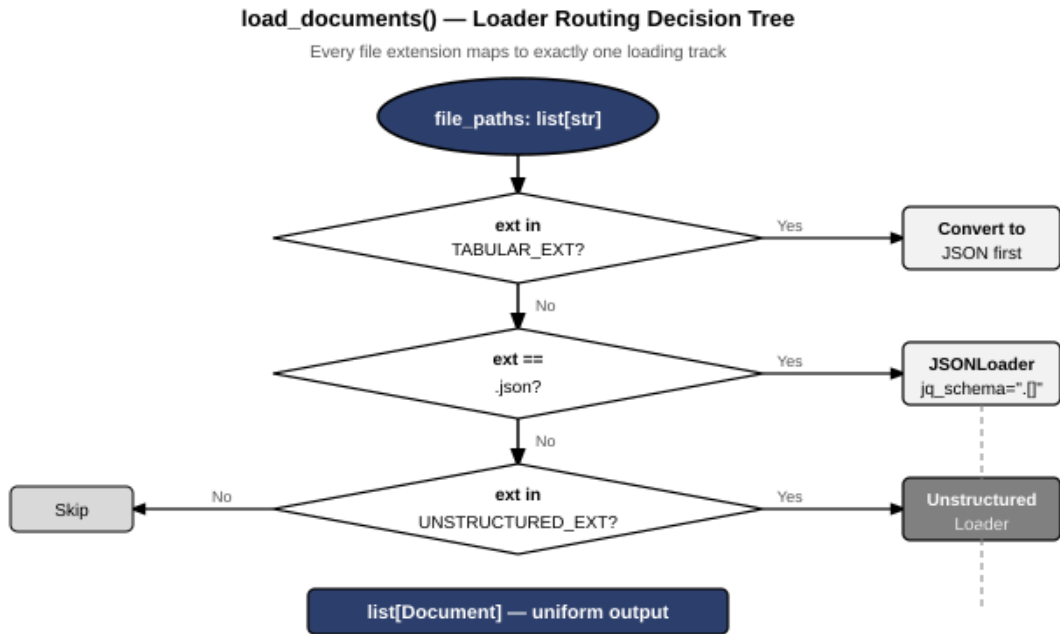


Figure 10.3 — Loader routing in `load_documents()`. The extension-based decision tree maps every supported file type to exactly one loading track, and all tracks converge on the same `List[Document]` output.

10.4 Splitting, Embedding, and Persisting in One Run

With a flat `List[Document]` in hand, the `main()` function calls three more steps in sequence: `split_documents()`, `EmbeddingManager.generate_embedding()`, and `VectorStore.add_documents_in_batches()`. Each step has been described in isolation in previous chapters; this section focuses on how they compose in a single run.

Splitting

`split_documents()` passes every `Document` through a `RecursiveCharacterTextSplitter` configured at `CHUNK_SIZE=1000` and `CHUNK_OVERLAP=200`. After splitting, it stamps a `chunk_seq` integer onto each chunk's metadata, resetting the counter per source file. This sequence number is what enables Neighbor-Aware Compression (Chapter 10's companion feature) to identify and merge adjacent chunks from the same document at query time. It also filters out stub chunks — any chunk shorter than `MIN_CHUNK_CHARS=150` characters — which arise from document headers, footers, and empty section breaks that carry no retrievable content.

```
def split_documents(documents: list[Document]) -> list[Document]:
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=CHUNK_SIZE,          # 1000 chars
        chunk_overlap=CHUNK_OVERLAP,    # 200 chars
        length_function=len,
        separators=["\n\n", "\n", " ", ""],
    )
    chunks = splitter.split_documents(documents)

    # Stamp chunk_seq per source so adjacent chunks are identifiable
    seq_counter: dict[str, int] = defaultdict(int)
    for chunk in chunks:
        src = chunk.metadata.get("source", "__unknown__")
        chunk.metadata["chunk_seq"] = seq_counter[src]
        seq_counter[src] += 1

    # Filter stub chunks
    before = len(chunks)
    chunks = [c for c in chunks if len(c.page_content.strip()) >=
MIN_CHUNK_CHARS]
    print(f"Filtered {before - len(chunks)} stub chunks. {len(chunks)}
remain.")
    return chunks
```

Embedding

`split_documents()` returns a list of `Document` objects. `VectorStore.add_documents_in_batches()` — which is responsible for both embedding and persisting — extracts the `page_content` strings in batches of `BATCH_SIZE=512`, passes them to `EmbeddingManager.generate_embedding()`, and receives back a NumPy array of L2-normalised float32 vectors. The batch size of 512 is large enough to fully utilise a GPU's parallelism without exceeding typical VRAM limits. Each batch's embeddings are then handed to `add_document()` for ChromaDB insertion before the next batch is processed, keeping peak memory usage bounded.

Persisting

`add_document()` calls `collection.add()` with four parallel lists: `ids`, `embeddings`, `metadatas`, and `documents`. In the current implementation, IDs are generated by concatenating a UUID4 with a batch index. This is fast and simple, but it makes the IDs non-deterministic — meaning a second run will generate different IDs for the same chunks, which is the root cause of duplicate entries on re-run. Section 10.6 discusses how to replace UUID-based IDs with deterministic content hashes to close this gap.

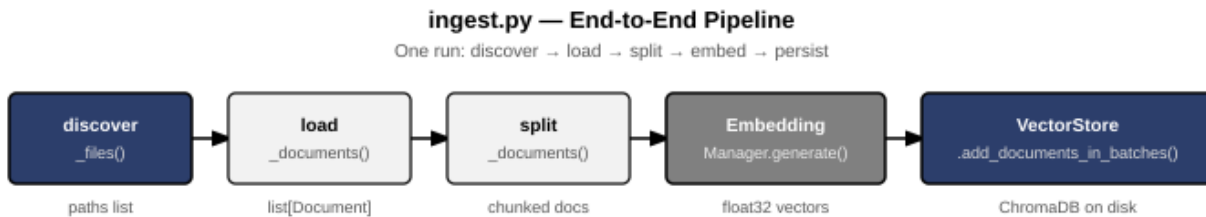


Figure 10.4 — `ingest.py` end-to-end pipeline. Five functions compose into a single run: `discover`, `load`, `split`, `embed`, `persist`. Each function's output is the next function's input.

10.5 Logging, Progress Bars, and Idempotency

A production ingestion script needs to be observable. When it runs silently and finishes with "Done," you have no idea how many files were processed, whether any failed, or how many chunks ended up in the store. When something goes wrong — and something always goes wrong — you need enough information in the logs to diagnose the failure without re-running the entire pipeline.

What to log and when

The current implementation logs at four natural checkpoints: after file discovery (how many files were found), after loading (how many document sections were produced and from how many files), after splitting (how many chunks remain after filtering), and after persisting (the total document count in the collection). These four numbers form a quick sanity check: if any of them is zero, ingestion has silently failed at that stage.

```

# Discovery checkpoint
print(f"Found {len(file_paths)} supported files")

# Loading checkpoint
print(f"UnstructuredLoader: {len(docs)} sections from
{len(unstructured_paths)} files")
print(f"Loaded {len(all docs)} document pages/sections total")

# Splitting checkpoint
print(f"Split {len(documents)} documents into {len(chunks)} chunks")
print(f"Filtered {before - len(chunks)} stub chunks. {len(chunks)}
remain.")

# Persistence checkpoint
print(f"Total documents in store: {vector_store.collection.count()}")
  
```

Progress indication during embedding

The embedding step is the most time-consuming part of ingestion. On a CPU, embedding 1000 chunks of 1000 characters each can take several minutes. Without a progress indicator, the user sees a frozen terminal and has no way to estimate how much longer the run will take. The sentence-transformers library's `encode()` method accepts a `show_progress_bar` parameter that renders a tqdm bar with ETA. During batch ingestion, set it to `False` (because the bar would reset on every batch call, producing a confusing cascade of 100% completions) and instead print a per-batch progress line manually:

```
for start in range(0, total, batch_size):
    end = min(start + batch_size, total)
    print(f"Processing batch {start} - {end - 1}")
    batch_embeddings = embedding_manager.generate_embedding(
        batch_texts[start:end],
        batch_size=batch_size,
        show_progress_bar=False, # suppress per-batch reset
    )
    self.add_document(batch_docs, batch_embeddings)
```

Definition — Idempotency

A property of an operation such that running it once produces the same result as running it ten times. An idempotent `ingest.py` can be re-run at any time — after adding a new file, after fixing a bug, after a crash mid-run — without producing duplicate entries or corrupting existing data. Idempotency does not mean the operation is free; it means the result is always correct regardless of how many times it has already been run.

10.6 Re-Running Ingestion Without Duplicating Entries

The most important property of a production ingestion script is the ability to re-run it safely. New documents arrive. Chunking parameters change. A bug in a loader gets fixed. In all of these cases, you need to re-ingest — and re-ingestion must not double every entry already in the store.

Two strategies address this problem, and they make different trade-offs.

Strategy A — Clear and Re-Ingest

The simplest approach is to delete the collection entirely before running and then re-ingest all documents from scratch. This guarantees a clean slate: no duplicates, no stale chunks from deleted source files, no residue from failed previous runs. The implementation is two lines at the top of `main()`:

```
def main():
    # Strategy A: clear before re-ingesting
    client = chromadb.PersistentClient(path=VECTOR_STORE_PATH)
    try:
        client.delete_collection(COLLECTION_NAME)
        print(f"Cleared existing collection '{COLLECTION_NAME}'")
    except Exception:
        pass # collection did not exist yet — that's fine

    vector_store = VectorStore(
        collection_name=COLLECTION_NAME,
        persist_directory=VECTOR_STORE_PATH,
    )
    # ... rest of ingestion unchanged
```

The trade-off: Strategy A deletes the entire collection. If your system uses a single ChromaDB instance for both the ingested documents and the `learned_qa` collection that accumulates verified Q&A pairs from the self-learning loop, you must ensure you are only deleting the documents collection, not the `learned_qa` collection. The two collections should always be kept separate for precisely this reason.

Strategy B — Deterministic IDs

The second approach leaves the collection intact and instead generates chunk IDs that are deterministic — derived from the chunk's content and position rather than from a random UUID. If a chunk with that ID already exists in the collection, ChromaDB's `upsert()` call either inserts it (first run) or overwrites it (subsequent runs) without creating a second entry. The formula for a deterministic ID is a hash of the source path and the chunk sequence number:

```
import hashlib

def make_chunk_id(source: str, chunk_seq: int) -> str:
    """Deterministic ID stable across re-runs."""
    raw = f"{source}::{chunk_seq}"
    return hashlib.md5(raw.encode()).hexdigest()

# In add_document(), replace the UUID line:
# BEFORE: ids.append(str(uuid.uuid4()) + f" {i}")
# AFTER:
for i, (doc, embedding) in enumerate(zip(documents, embeddings)):
    src = doc.metadata.get("source", " unknown ")
    seq = doc.metadata.get("chunk_seq", i)
    chunk_id = _make_chunk_id(src, seq)
    ids.append(chunk_id)

# Replace collection.add() with collection.upsert():
self.collection.upsert(
    ids=ids,
    embeddings=embeddings_list,
    metadatas=metadatas,
    documents=documents_text,
)
```

Strategy B is preferable when re-ingestion is incremental — when only a subset of files changes on each run. It allows the unchanged chunks to pass through without touching ChromaDB (their IDs already exist; the `upsert` is a no-op), and only the changed or new chunks are actually written. The cost is that stale chunks from deleted source files are not removed automatically; a separate cleanup pass is required to remove chunks whose source file no longer exists on disk.

Common pitfall — `upsert()` does not remove stale chunks

If you delete a source file from disk and re-run ingestion with Strategy B, its chunks remain in the vector store forever. They will continue to surface at retrieval time, potentially returning answers from a document that no longer exists. The fix is to build an explicit cleanup step: after discovery, collect the set of currently-seen source paths; query ChromaDB for all existing source values; and delete any chunk whose source does not appear in the current file set.

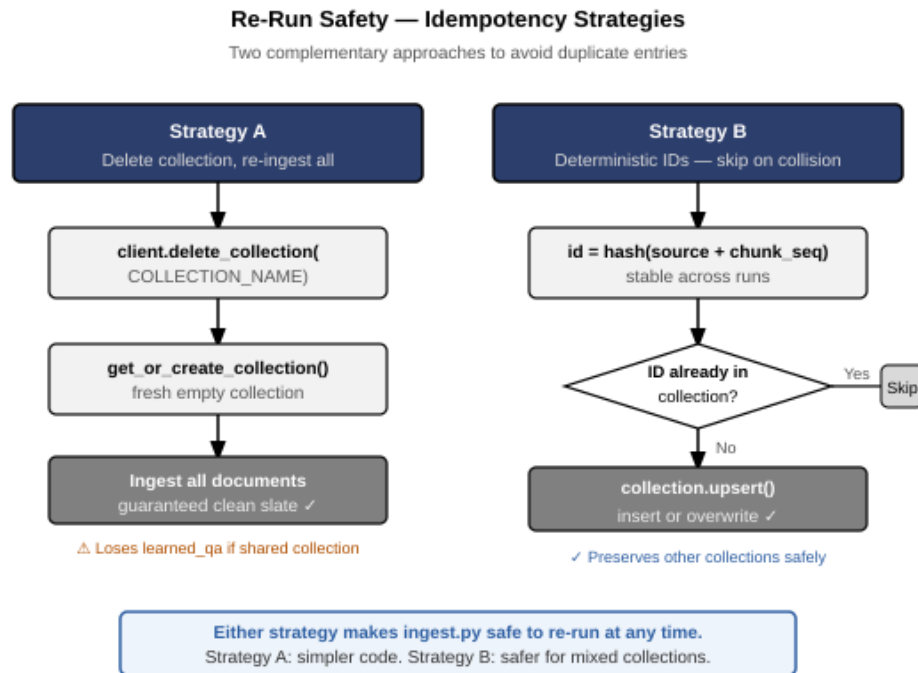


Figure 10.5 — Re-run safety strategies. Strategy A clears the collection for a guaranteed clean slate. Strategy B uses deterministic content-hash IDs with `upsert()` to make repeated runs idempotent without deleting anything.

Skeleton Code — ingest.py

The following is the minimal, annotated skeleton of `ingest.py`. Each function is shown with its essential logic and the configuration constants it depends on. Extend by adding new extensions to `SUPPORTED_EXT` and adding a corresponding branch in `load_documents()`.

```
# ingest.py - Skeleton
from pathlib import Path
from collections import defaultdict
from langchain_core.documents import Document
from langchain.text splitters import RecursiveCharacterTextSplitter
from langchain_unstructured import UnstructuredLoader
from langchain_community.document_loaders import JSONLoader
from embedding_manager import EmbeddingManager
from vector_store import VectorStore
import json, pandas as pd

# — Configuration —————
UNSTRUCTURED_EXT = {".pdf", ".txt", ".doc", ".docx", ".html", ".htm"}
TABULAR_EXT      = {".csv", ".xls", ".xlsx"}
SUPPORTED_EXT    = UNSTRUCTURED_EXT | TABULAR_EXT | {".json"}
SEARCH_ROOTS     = [Path("../data/pdfs"), ..., Path("../data")]
CHUNK_SIZE, CHUNK_OVERLAP, MIN_CHUNK_CHARS = 1000, 200, 150
BATCH_SIZE       = 512
VECTOR_STORE_PATH = "../data/vector_store"
COLLECTION_NAME  = "documents"

# — 1. File discovery —————
def discover_files() -> list[str]: ...

# — 2. Tabular conversion —————
def _convert_tabular_to_json(p: Path) -> Path | None: ...

# — 3. Loader routing —————
def load_documents(file_paths: list[str]) -> list[Document]: ...

# — 4. Splitting + chunk_seq stamping —————
def split_documents(documents: list[Document]) -> list[Document]: ...

# — 5. Orchestration —————
def main():
    file_paths = discover_files()
    documents  = load_documents(file_paths)
    chunks     = split_documents(documents)
    em         = EmbeddingManager()
    vs         = VectorStore(COLLECTION_NAME, VECTOR_STORE_PATH)
    vs.add_documents_in_batches(chunks, em, batch_size=BATCH_SIZE)
    print(f"Total documents in store: {vs.collection.count()}")

if __name__ == "__main__":
    main()
```